

CS-548 - Cloud-native Software Architectures

Assignment A1: Docker

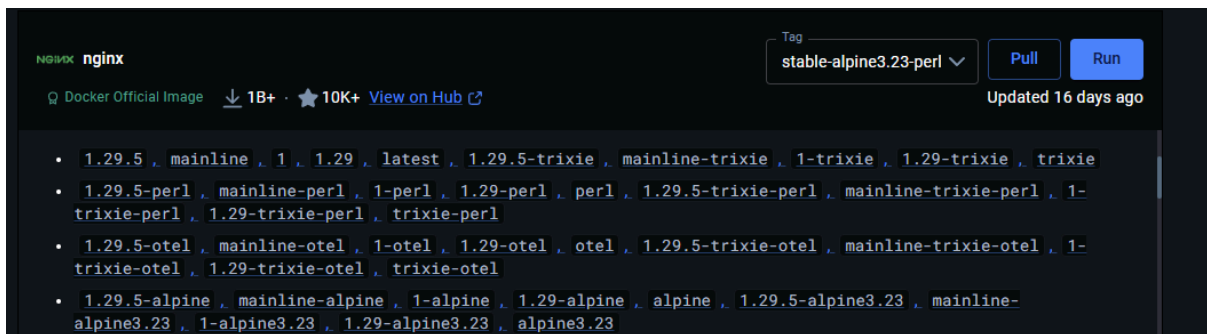
Μάνος Αλέξανδρος csd-5136

Due Date: 04/03/2026

Άσκηση 1

a)

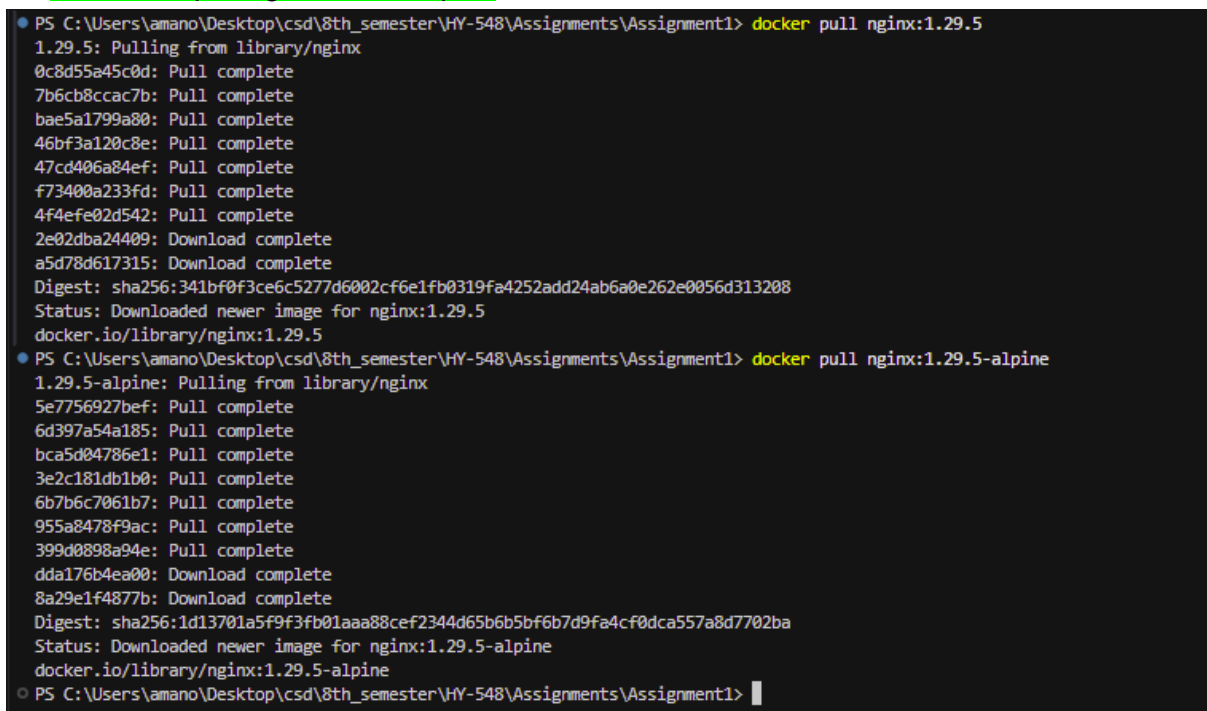
Μπορούμε να κατεβάσουμε τα images που θέλουμε από την εφαρμογή docker desktop:



αλλά εφόσον πρέπει να το κάνουμε από το terminal, θα χρησιμοποιήσουμε την εντολή docker pull.

Συγκεκριμένα τις εντολές:

- `docker pull nginx:1.29.5`
- `docker pull nginx:1.29.5-alpine`



b)

Η εντολή `docker images` μας επιστρέφει την λίστα με όλα τα images. (Πέρα από το nginx, βλέπουμε και άλλα images από διάφορα tutorials του docker desktop ή από το youtube).

```
PS C:\Users\amano\Desktop\csd\8th_semester\HY-548\Assignments\Assignment1> docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
bindmount-apps-todo-app:latest	9a416dd1352a	278MB	58.3MB	U
docker/welcome-to-docker:latest	c4d56c24da4f	22.2MB	6.03MB	U
example:latest	f1a3135145d4	1.64GB	410MB	
mongo:6	03cda579c8ca	1.06GB	273MB	U
multi-container-app-todo-app:latest	6c5129e2806f	325MB	65.3MB	U
nginx:1.29.5	341bf0f3ce6c	240MB	65.8MB	
nginx:1.29.5-alpine	1d13701a5f9f	93.4MB	26.9MB	
nygma13/welcome-to-docker:latest	c4d56c24da4f	22.2MB	6.03MB	U
welcome-to-docker:latest	89b8e14cfacc	483MB	131MB	U

```
PS C:\Users\amano\Desktop\csd\8th_semester\HY-548\Assignments\Assignment1>
```

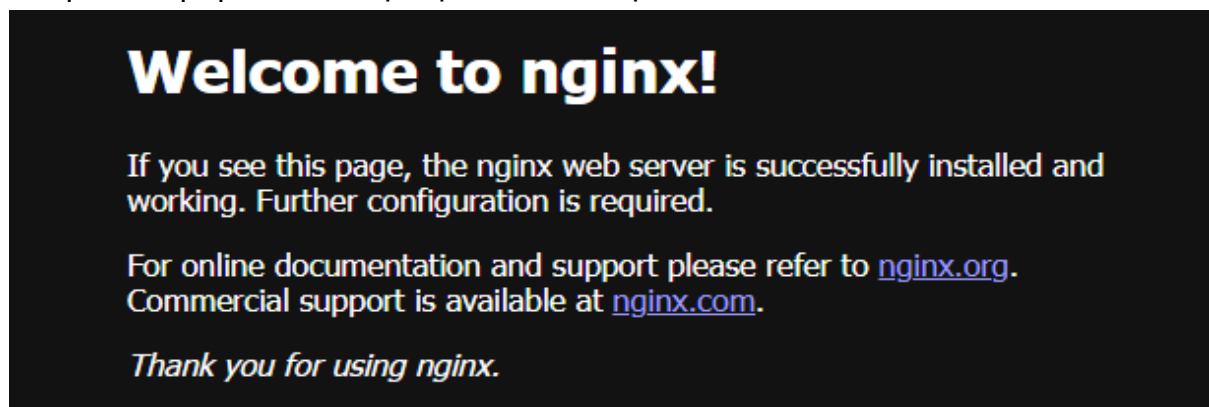
Αμέσως παρατηρούμε την διαφορά μεγέθους των δύο images. Το nginx:1.29.5 είναι 240MB ενώ το nginx:1.29.5-alpine είναι 93.4MB. Αυτή η διαφορά οφείλεται στο γεγονός ότι το alpine είναι ένα από τα πιο lightweight Linux distributions καθώς περιέχει τα απαραίτητα εργαλεία (βιβλιοθήκες κλπ.) για να τρέξει το nginx, ενώ το κανονικά nginx είναι πιο μεγάλο διότι περιέχει πολλά pre-installed tools (βιβλιοθήκες κ.α.).

c)

Για να ξεκινήσουμε το nginx στο background, θα χρησιμοποιήσουμε την εντολή `docker run`, με flags/options `-d` (start a container in background), `-p` (map a port) με τα δύο “ορίσματα” `HOST_PORT:CONTAINER_PORT`. Συνολικά η εντολή που θα χρησιμοποιήσουμε είναι `docker run -d -p 8000:80 nginx:1.29.5`.

```
PS C:\Users\amano\Desktop\csd\8th_semester\HY-548\Assignments\Assignment1> docker run -d -p 8000:80 nginx:1.29.5
705af2f2ca9825c0f5de4f79dc3cdc10f90b91484c8bddfb1fa5cd299aa38e07
```

και η απάντηση που θα πάρουμε από το “http://localhost:8000/”:



d)

Για να δούμε ότι τρέχει το container χρησιμοποιούμε την εντολή: `docker ps` → shows running containers

```
PS C:\Users\amano\Desktop\csd\8th_semester\HY-548\Assignments\Assignment1> docker run -d -p 8000:80 nginx:1.29.5
705af2f2ca9825c0f5de4f79dc3cdc10f90b91484c8bddfb1fa5cd299aa38e07
PS C:\Users\amano\Desktop\csd\8th_semester\HY-548\Assignments\Assignment1> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
705af2f2ca98	nginx:1.29.5	"/docker-entrypoint..."	12 minutes ago	Up 12 minutes	0.0.0.0:8000->80/tcp, [::]:8000->80/tcp	sad_borg

e)

Για τα logs του τρέχον container χρησιμοποιούμε την εντολή `docker logs` ακολουθούμενο από το ID του container. Συγκεκριμένα: `docker logs 705af2f2ca98`

```
PS C:\Users\amano\Desktop\csd\8th_semester\HY-548\Assignments\Assignment1> docker logs 705af2f2ca98
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/02/24 14:37:09 [notice] 1#1: using the "epoll" event method
2026/02/24 14:37:09 [notice] 1#1: nginx/1.29.5
2026/02/24 14:37:09 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/02/24 14:37:09 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-65L2
2026/02/24 14:37:09 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576;1048576
2026/02/24 14:37:09 [notice] 1#1: start worker processes
2026/02/24 14:37:09 [notice] 1#1: start worker process 29
2026/02/24 14:37:09 [notice] 1#1: start worker process 30
2026/02/24 14:37:09 [notice] 1#1: start worker process 31
2026/02/24 14:37:09 [notice] 1#1: start worker process 32
2026/02/24 14:37:09 [notice] 1#1: start worker process 33
2026/02/24 14:37:09 [notice] 1#1: start worker process 34
2026/02/24 14:37:09 [notice] 1#1: start worker process 35
2026/02/24 14:37:09 [notice] 1#1: start worker process 36
2026/02/24 14:37:09 [notice] 1#1: start worker process 37
2026/02/24 14:37:09 [notice] 1#1: start worker process 38
2026/02/24 14:37:09 [notice] 1#1: start worker process 39
2026/02/24 14:37:09 [notice] 1#1: start worker process 40
172.17.0.1 - - [24/Feb/2026:14:37:44 +0000] "GET / HTTP/1.1" 200 615 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36" "-"
172.17.0.1 - - [24/Feb/2026:14:37:44 +0000] "GET /favicon.ico HTTP/1.1" 404 555 "http://localhost:8000/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36" "-"
2026/02/24 14:37:44 [error] 30#30: *1 open() "/usr/share/nginx/html/favicon.ico" failed (2: No such file or directory), client: 172.17.0.1, server: localhost, request: "GET /favicon.ico HTTP/1.1", host: "localhost:8000", referer: "http://localhost:8000/"
```

f)

Για να σταματήσουμε το running container: `docker stop 705af2f2ca98`

g)

Για να ξεκινήσουμε το stopped container: `docker start 705af2f2ca98`

h)

Για να σταματήσουμε το container και να το διαγράψουμε, ή θα κάνουμε σειριακά `docker stop` και `docker rm` ή `docker rm -f` όπου με αυτό το flag διαγράφουμε ένα running container. Συγκεκριμένα `docker rm -f 705af2f2ca98`

```
PS C:\Users\amano\Desktop\csd\8th_semester\HY-548\Assignments\Assignment1> docker stop 705af2f2ca98
705af2f2ca98
PS C:\Users\amano\Desktop\csd\8th_semester\HY-548\Assignments\Assignment1> docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
PS C:\Users\amano\Desktop\csd\8th_semester\HY-548\Assignments\Assignment1> docker start 705af2f2ca98
705af2f2ca98
PS C:\Users\amano\Desktop\csd\8th_semester\HY-548\Assignments\Assignment1> docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
705af2f2ca98   nginx:1.29.5   "/docker-entrypoint..."   28 minutes ago   Up 1 second   0.0.0.0:8000->80/tcp, [::]:8000->80/tcp   sad_borg
PS C:\Users\amano\Desktop\csd\8th_semester\HY-548\Assignments\Assignment1> docker rm -f 705af2f2ca98
705af2f2ca98
PS C:\Users\amano\Desktop\csd\8th_semester\HY-548\Assignments\Assignment1> docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
PS C:\Users\amano\Desktop\csd\8th_semester\HY-548\Assignments\Assignment1>
```

Άσκηση 2

a)

Ξανά ξεκινάμε το container (όχι start) με την εντολή `docker run -d -p 8000:80 nginx:1.29.5`. Τώρα για να ανοίξουμε shell session μέσα σε running container χρησιμοποιούμε την εντολή `docker exec -it b0453179f357 bin/bash`.

```
PS C:\Users\amano\Desktop\csd\8th_semester\HY-548\Assignments\Assignment1> docker exec -it b0453179f357 bin/bash
root@b0453179f357:/# ls usr/share/nginx/
html
root@b0453179f357:/# ls usr/share/nginx/html/
50x.html index.html
root@b0453179f357:/# vim usr/share/nginx/html/index.html
bash: vim: command not found
root@b0453179f357:/# nano usr/share/nginx/html/index.html
bash: nano: command not found
root@b0453179f357:/# cat usr/share/nginx/html/index.html
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
root@b0453179f357:/# sed -i 's/Welcome to nginx!/Hello Alex/' /usr/share/nginx/html/index.html
root@b0453179f357:/# vi usr/share/nginx/html/index.html
bash: vi: command not found
root@b0453179f357:/# exit
exit
```

Μετά την εντολή `sed -i 's/Welcome to nginx!/Hello Alex/' /usr/share/nginx/html/index.html` (και φυσικά ένα refresh στο page) παρατηρούμε ότι ο τίτλος (first sentence) άλλαξε σε:

Hello Alex

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

b)

Έκανα copy το html αρχείο από το container, τοπικά, αλλά επειδή το πήρα από το container που είχα ήδη αλλάξει την πρώτη πρόταση από το ερώτημα a (για να πάρω το default μπορούσα να ξεκινήσω άλλο container και να το τραβήξω από εκεί), το άλλαξα με το χέρι μέσω του vs code και έβαλα κάτι διαφορετικό για να ξεχωρίζει. Για το copy από το container-τοπικά χρησιμοποίησα την εντολή:

```
docker cp b0453179f357:/usr/share/nginx/html/index.html .
```

ενώ από τοπικά για να το ανεβάσω στο container:

```
docker cp index.html b0453179f357:/usr/share/nginx/html/index.html
```



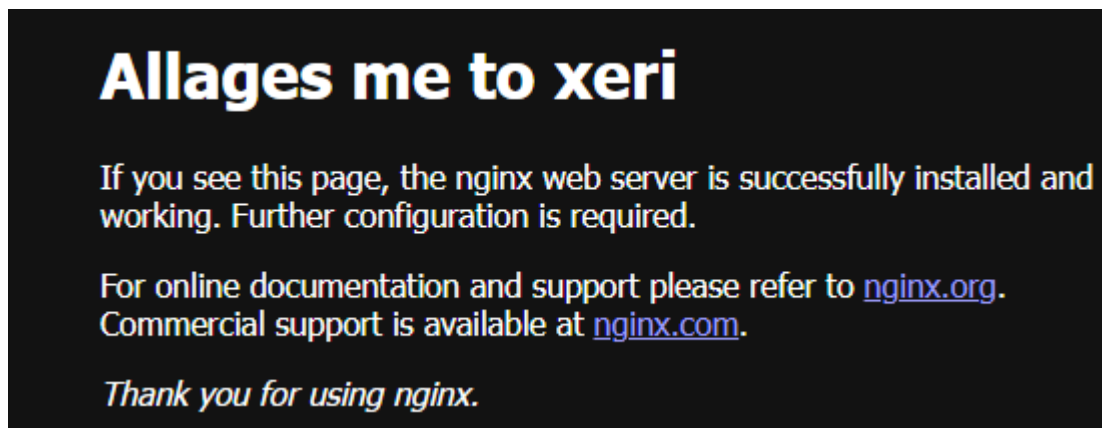
The screenshot shows a VS Code editor with a file named `index.html` containing the following HTML code:

```
<body>
  <h1>Allages me to xeri</h1>
  <p>If you see this page, the nginx web server is successfully installed and
    working. Further configuration is required.</p>
  <p>For online documentation and support please refer to
    <a href="http://nginx.org/">nginx.org</a>.<br />
    Commercial support is available at
    <a href="http://nginx.com/">nginx.com</a>.</p>
  <p><em>Thank you for using nginx.</em></p>
</body>
```

Below the editor, the terminal shows the following commands and output:

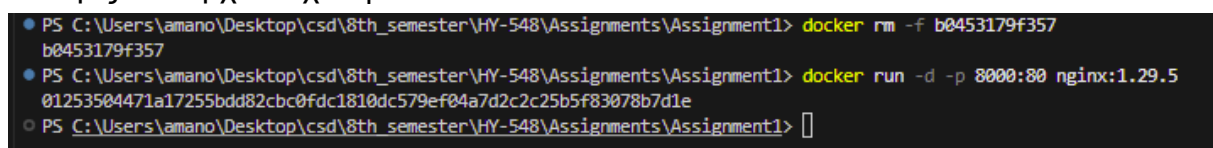
```
PS C:\Users\amano\Desktop\csd\8th_semester\HY-548\Assignments\Assignment1> docker cp b0453179f357:/usr/share/nginx/html/index.html .
Successfully copied 2.56kB to C:\Users\amano\Desktop\csd\8th_semester\HY-548\Assignments\Assignment1\
PS C:\Users\amano\Desktop\csd\8th_semester\HY-548\Assignments\Assignment1> docker cp index.html b0453179f357:/usr/share/nginx/html/index.html
Successfully copied 2.56kB to b0453179f357:/usr/share/nginx/html/index.html
PS C:\Users\amano\Desktop\csd\8th_semester\HY-548\Assignments\Assignment1> 
```

Το αποτέλεσμα:



c)

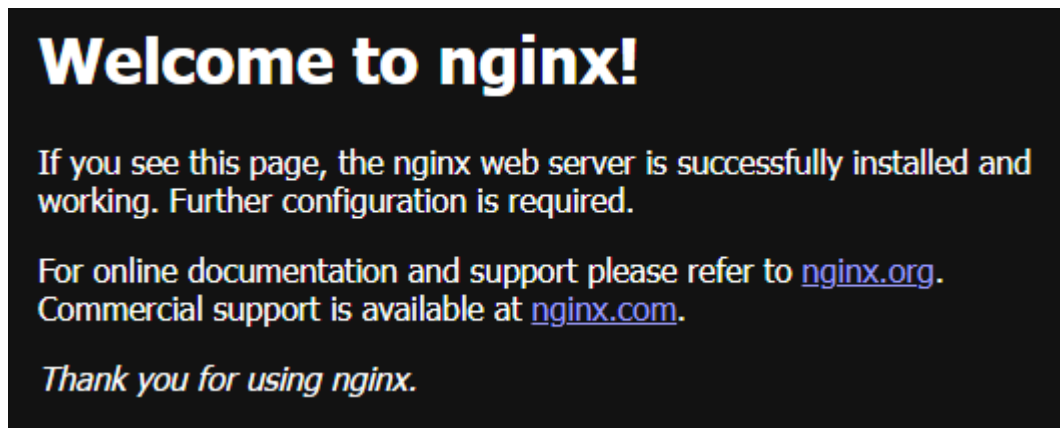
Αφού διαγράψω και ξανα ξεκινήσω (docker run) άλλο container παρατηρούμε ότι οι αλλαγές του αρχείου χάθηκαν.



The screenshot shows the following terminal commands and output:

```
PS C:\Users\amano\Desktop\csd\8th_semester\HY-548\Assignments\Assignment1> docker rm -f b0453179f357
b0453179f357
PS C:\Users\amano\Desktop\csd\8th_semester\HY-548\Assignments\Assignment1> docker run -d -p 8000:80 nginx:1.29.5
01253504471a17255bdd82cbc0fdc1810dc579ef04a7d2c2c25b5f83078b7d1e
PS C:\Users\amano\Desktop\csd\8th_semester\HY-548\Assignments\Assignment1> 
```

και αυτό το βλέπουμε από το page:

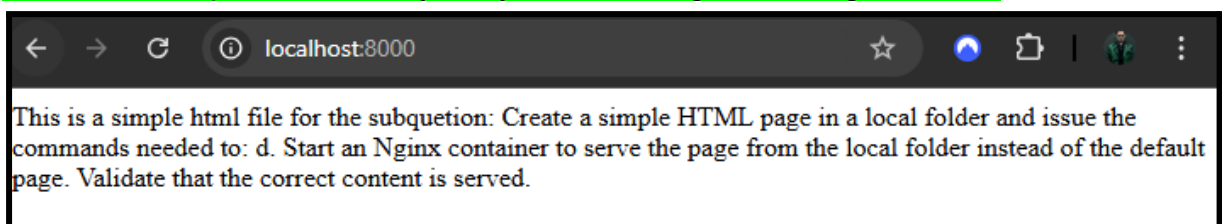


Αυτό συμβαίνει διότι τα images είναι R/O layers. Το R/W layer προστίθεται κατά το runtime, όπου αποθηκεύτηκαν οι αλλαγές που έκανα. Όταν διέγραφα το προηγούμενο container, διαγράφηκε και το layer αυτό. (Για να κρατήσουμε τις αλλαγές μπορούμε να κάνουμε commit το image μας ή να χρησιμοποιήσουμε volumes).

d)

Για να κάνω map με το local directory μέσα στο container χρησιμοποίησα την εντολή:

```
docker run -d -p 8000:80 -v ${PWD}:/usr/share/nginx/html nginx:1.29.5
```



(Note: τα commands γράφω σε powershell, οπότε χρησιμοποίησα το \${PWD}).

Άσκηση 3

a)

Για το package vim-tiny προσθέτουμε στο Dockerfile το εξής:

```
RUN apt-get update && \
    apt-get install -y vim-tiny && \
    rm -rf /var/lib/apt/lists/*
```

Αποφεύγουμε έτσι να κάνουμε πολλαπλά layers για ένα πράγμα + ότι αδειάζουμε τις λίστες (make it smaller).

Για να κρατήσουμε το database σε ξεχωριστό volume μπορούμε να δημιουργήσουμε ένα compose.yaml αρχείο και να διαχωρίσουμε το app volume από το data base volume. Ωστόσο μπορούμε να το προσθέσουμε στο Dockerfile:

VOLUME /app/db

(Note: προσθέτουμε το db γιατί σε αυτό το directory αποθηκεύεται το database file.)

Το κάνω build με την εντολή: `docker build -t assignment1-exerc3 .`

Επειδή δε μου εμφανίζει το based image (python:3.13.2-bookworm) το έκανα pull.

```
PS C:\Users\amano\Desktop\csd\8th_semester\HY-548\Assignments\Assignment1\exerc_3\django> docker pull python:3.13.2-bookworm
3.13.2-bookworm: Pulling from library/python
Digest: sha256:4165118ed569aff9dbd11d5518199e5379d93bf5bf1cdda62eb13593cf66fb68
Status: Downloaded newer image for python:3.13.2-bookworm
docker.io/library/python:3.13.2-bookworm
PS C:\Users\amano\Desktop\csd\8th_semester\HY-548\Assignments\Assignment1\exerc_3\django> docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
assignment1-exerc3:latest	9f7b246b62e7	1.55GB	395MB	
bindmount-apps-todo-app:latest	9a416dd1352a	278MB	58.3MB	U
docker/welcome-to-docker:latest	c4d56c24da4f	22.2MB	6.03MB	U
example:latest	f1a3135145d4	1.64GB	410MB	
mongo:6	03cda579c8ca	1.06GB	273MB	U
multi-container-app-todo-app:latest	6c5129e2806f	325MB	65.3MB	U
nginx:1.29.5	341bf0f3ce6c	240MB	65.8MB	U
nginx:1.29.5-alpine	1d13701a5f9f	93.4MB	26.9MB	
nygm13/welcome-to-docker:latest	c4d56c24da4f	22.2MB	6.03MB	U
python:3.13.2-bookworm	4165118ed569	1.49GB	398MB	
welcome-to-docker:latest	89b8e14cfacc	483MB	131MB	U

```
PS C:\Users\amano\Desktop\csd\8th_semester\HY-548\Assignments\Assignment1\exerc_3\django>
```

Βλέπουμε ότι το image που δημιούργησα (assignment1-exerc3) είναι 1.55 GB ενώ το based image είναι 1.49 GB. Αυτό οφείλεται στο γεγονός ότι στο Dockerfile πρόσθεσα καινούριες εντολές (+ οι extra που υπήρχαν από το github), αρα πρόσθεσα ουσιαστικά καινούρια R/O layers, με αποτέλεσμα να αυξηθεί το μέγεθος του image μου.

b)

Μπέρδεψα το όνομα του container που έχει “απενεργοποιημένο” το debug και το ονόμασα debug-app.

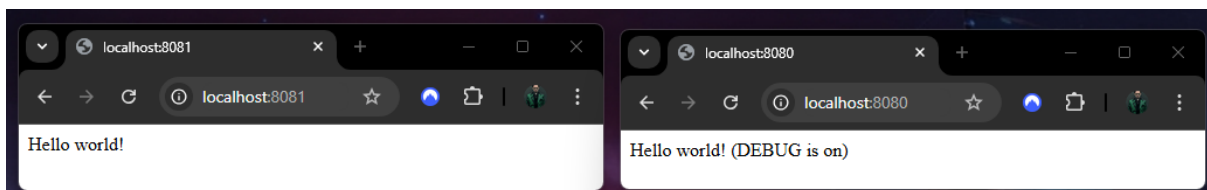
Για το container με debug mode έτρεξα την εντολή:

```
docker run -d -p 8080:8000 --name def-app 9f7b246b62e7
```

ενώ για το container without debug mode:

```
docker run -d -p 8081:8000 -e DJANGO_DEBUG=0 --name debug-app  
9f7b246b62e7
```

Τα αποτελέσματα:

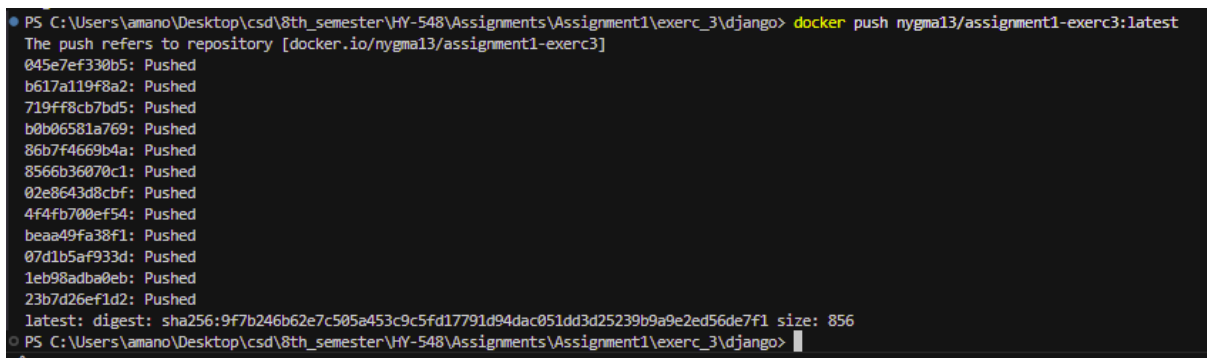


(Note: χρησιμοποίησα διαφορετικά ports για να μην υπάρχει conflict.)

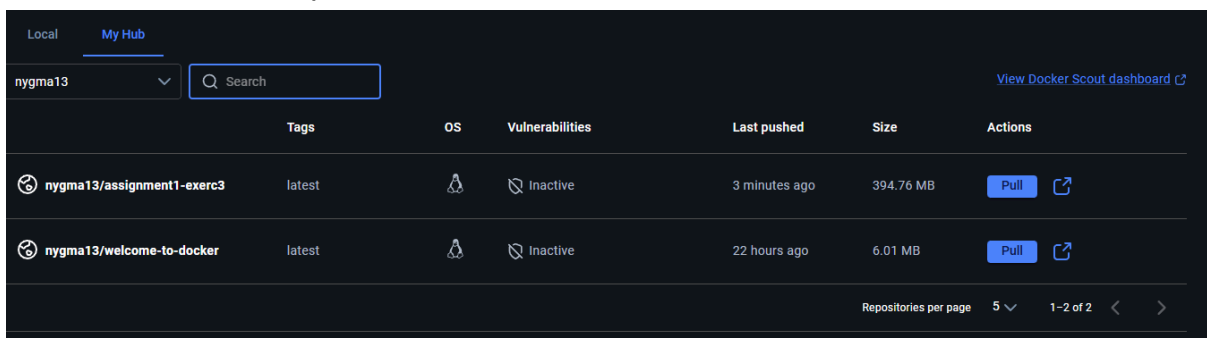
c)

Έβαλα tag στο image μου, έκανα docker login και μετά έτρεξα την εντολή:

```
docker push nygma13/assignment1-exerc3:latest
```

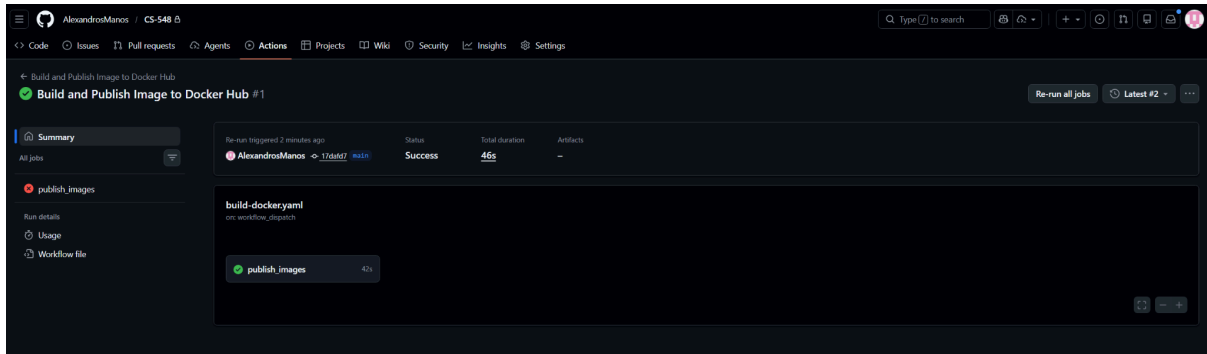


Από το docker desktop:



Άσκηση 4

Μιας και δε βρήκα κάτι σχετικό στις διαφάνειες, ακολούθησα το [tutorial](#) που βρήκα στο youtube.



Έκανα generate token για το docker, όρισα “Secrets” το token αυτό μέσω github settings με όνομα DOCKER_HUB_TOKEN και έβαλα στο build-docker.yaml το όνομα μου από το Docker (“nygma13”) και για password (-p) το token αυτό. Μπορούσα να ορίσω και το username σαν variable.

